Requisitos Funcionais (RF)

RF 01 - Game Loop em Terminal: Execução contínua do loop de jogo em C (leitura de input, atualização de estado, impressão de tela em console).

RF 02 - Navegação entre Salas: Sistema de movimentação modularizado em 3 blocos (Bloco 1: Tutorial, Bloco 2: Exploração/Puzzles de Alucinação, Bloco 3: Boss Final).

RF 03 - Gerenciamento de Estado do Jogador: Controle e exibição de vidas, inventário de itens, nível de pureza/poder dos itens e contador de ações.

RF 04 - Mecânica de Puzzles e Janela de Contexto: Apresentação de dilemas éticos/conceituais sobre IA com penalidade/recompensa baseada no número de ações e taxa de acertos.

RF 05 - Comportamento Dinâmico do Companion/Boss: Atualização do estado do companion (amigável vs. agressivo/danificado) baseado nos erros do jogador e transição para confronto final.

RF 06 - Integração via FFI (Onda 2): Invocação de regras puras e validações em Haskell a partir do runtime em C.

RF 07 - Persistência (Onda 2): Leitura e escrita de progresso em arquivo texto (.txt).

Requisitos Não Funcionais (RNF)

RNF 01 - Gerenciamento de Memória em C: Ausência de vazamento de memória (*memory leaks*) nas alocações dinâmicas de buffers de texto e structs de estado.

RNF 02 - Portabilidade: Compilação multiplataforma (Linux e Windows) via GCC (e GHC na U2).

RNF 03 - Usabilidade de Console: Interface em modo texto limpa, com tempo de resposta imediato a comandos de teclado e menus legíveis.

Restrições Técnicas (REST)

REST 01 - Ecossistema Estrito: C padrão (U1) e integração com Haskell via FFI (U2), sem uso de engines visuais ou banco de dados externo.

REST 02 - Fronteira FFI: Troca de dados restrita a tipos primitivos compatíveis com C (CInt, CString, ponteiros opacos).